

SMART INDIA HACKATHON 2026

Project AEGIS

Automated Emergency Guidance & Intelligence Support

A five-stage, human-in-the-loop AI pipeline that turns a disaster image into a safe, prioritised rescue dispatch.

5

PIPELINE STAGES

100

ZONES PER IMAGE

2

HUMAN APPROVAL
GATES

3+

CORE AI / CV MODELS

Problem Statement ID	26206
Problem Statement Title	Student Innovation – Disaster Management (risk mitigation, planning and management before, after or during a disaster)
Theme	Disaster Management
PS Category	Software
Organization / Department	AICTE / AICTE, MIC – Student Innovation
Team Name	Team Diamonds

This document is a supporting project dossier prepared for internal team preparation — covering problem understanding, solution architecture, technology-stack rationale, datasets, judge-readiness talking points, anticipated Q&A, and future roadmap. It is meant to complement (not replace) the official SIH PPT submission.

CONTENTS

SECTION	PAGE
Executive Summary	3
Problem Understanding	4
Action Items — Fixes to Make Before Submission	4
The Problem, In Detail	6
Proposed Solution — AEGIS Pipeline	6
Frontend Decision: Next.js vs. Flutter	8
Datasets & References	10
Language That Signals Rigour to Judges	11
Future Enhancements	11
Anticipated Judge Questions & Suggested Answers	12
Feasibility, Viability & Scale — Summary	15

01 Executive Summary

AEGIS is a unified, human-in-the-loop AI system for flood-response coordination. It reads satellite or drone imagery of a disaster-hit area, splits it into a 100-zone grid, scores each zone's flood and damage severity, sends drones to verify the worst-hit zones, recommends the right mix of boats/ambulances/teams, and plans the safest route to get them there — with a human approving both the resource plan and the route before anything is dispatched. It targets **PS 26206 (Disaster Management)** by focusing on the response phase, while keeping its zone-severity data reusable for pre-disaster risk planning and post-disaster relief documentation as well.

02 Problem Understanding

What PS 26206 actually asks for: the statement is broad on purpose — "ideas related to risk mitigation, planning and management **before, after, or during** a disaster." It doesn't force one disaster type or one output format. A strong submission just needs to be clear about **which phase(s)** it targets and **why**, instead of trying to cover everything shallowly.

Where AEGIS sits: AEGIS is mainly a **during-and-after response coordination system**. It reads post-event imagery, scores each zone's severity, sends drones to the worst zones first, recommends resources, and plans safe dispatch routes — all with human approval gates. This is a strong, high-impact reading of the PS, because the first 24–72 hours of response is when most lives are lost or saved.

Strengthening the "before" angle: To show full-lifecycle thinking (recommended for judge Q&A even if not the core demo), position the zone-severity database as a historical layer that also powers pre-disaster planning:

- Historical zone severity + resource-response records build a **vulnerability index per zone**, usable for pre-monsoon risk-mitigation planning by district authorities.
- The same 10×10 grid + severity-scoring pipeline can run on **forecast / early-warning imagery** (IMD rainfall models, CWC flood forecasts) to pre-position resources before a flood peaks.
- Post-disaster after-action data (which routes failed, which resources were under/over-allocated) feeds back into **future planning playbooks** — closing the before→during→after loop.

Framing it this way costs nothing in the build, but answers the "does this only handle response, not mitigation/planning?" question before a judge asks it.

03 Action Items — Fixes to Make Before Submission

A short checklist to move this from a good idea to a fully-aligned, judge-ready submission. Each item is quick to add and directly answers a question a judge would otherwise ask.

#	Fix / Addition	Why it matters
1	Fill the blanks — Problem Statement ID, PS Title, and Team ID on slide 1.	The first thing evaluators check. A blank field can cause an administrative rejection, regardless of how good the idea is.
2	Show the "before" phase — add a line on using historical zone-severity data + IMD/CWC forecasts to build a pre-monsoon risk map for authorities.	PS 26206 covers risk mitigation and planning, not just response. This proves you covered the full scope of the PS, not just one phase of it.
3	Show the "after" phase — add a line on reusing severity data for post-disaster damage documentation (e.g. for insurance or government relief claims) and a rebuild-priority list.	Same underlying data, a new use — a low-effort way to cover the third phase ("before, after, or during") named in the PS.
4	Prove multi-hazard thinking — state clearly that swapping the training data (xBD covers floods, earthquakes, wildfires, hurricanes) lets the pipeline generalise beyond floods.	Judges often ask "does this only work for floods?" — this answers it before it's asked.
5	Show fit with government systems — one line stating the tool is designed to plug into NDMA/SDMA workflows as a decision-support layer, not replace them.	Judges worry about ideas that quietly assume they'll replace existing government infrastructure. Naming this removes that doubt.
6	Be upfront about build status — mark clearly which parts are real/working (e.g. vision analysis, severity scoring) versus simulated for the demo (e.g. drone dispatch, live SMS).	An accurate, confident scope holds up better under questioning than an implied claim that everything is production-ready.

#	Fix / Addition	Why it matters
7	Add a data-handling line — how victim/location data is stored and secured, and who can access it.	A common question in disaster-tech judging that is rarely addressed on any slide — answering it stands out.
8	Prioritise a live demo over polished slides — even a simple working flow (upload image → see zone scores → see a route) is worth more than perfect slides alone.	A working demo is usually the single biggest differentiator at SIH finals.

04 The Problem, In Detail

During a disaster, responders must rapidly understand flooding extent, structural damage, and affected areas — but ground-level assessment is slowed by floodwater, debris, and inaccessible terrain. Not every affected location needs the same response, yet with limited ambulances, boats, and rescue teams, resources are often deployed inefficiently. Roads and bridges damaged by flooding make conventional routing unsafe, and field teams need clear, timely, actionable instructions — not raw data.

Real-world anchor: the 2024–25 Bhote Koshi flash floods in Nepal illustrated exactly this — extensive damage to roads, bridges, homes, and critical infrastructure that overwhelmed manual coordination. This is used as the motivating case for AEGIS.

05 Proposed Solution — AEGIS Pipeline

AEGIS is a unified, human-in-the-loop AI pipeline that goes from a raw disaster image to a field-ready dispatch instruction in five stages:



Stage	What it does	Core tech
1. Vision Analysis	Ingests satellite/aerial imagery; performs flood segmentation and damage detection over the affected area.	U-Net (flood segmentation), YOLOv8 (damage/object detection), OpenCV
2. Zone Mapping & Prioritisation	Splits the image into a 10×10 (100-zone) grid; computes a flood score, damage score and combined severity score per zone; ranks zones high→low priority.	Custom scoring logic, SQLite zone database
3. Drone Intelligence	Deploys targeted drone passes only to top-priority zones; detects people, animals, and vehicles to build a victim-count estimate.	YOLOv8 object detection
4. AI Resource Decision	Combines severity, victim counts and disaster context to recommend required resources (boats, ambulances, rescue teams) per zone — gated by human approval.	Gemini API for reasoning, LangGraph orchestration
5. Safe Routing & Dispatch	Plans the shortest safe route avoiding flooded/blocked roads, then converts the approved plan into multilingual field instructions and alerts.	OpenStreetMap + Dijkstra (NetworkX), Twilio (SMS/WhatsApp), gTTS (audio), Google Translate API

Two human approval gates are deliberately built in — before resources are committed and before a route is dispatched — so the AI recommends but a human authority decides. This directly answers the "what if the AI is wrong" question judges commonly ask.

System Outputs

- **Flood map** — extent and depth estimation
- **Damage map** — detected structural damage
- **Priority zones** — ranked by severity score
- **Victim counts** — people, animals, vehicles detected
- **Resource plan** — recommended boats, ambulances, teams

- **Safe routes** — shortest safe paths with ETA and waypoints
- **Dispatch output** — SMS, WhatsApp, audio, multilingual instructions

06 Frontend Decision: Next.js vs. Flutter

This is one of the most common judge questions for a system with two very different user groups (a control-room authority and a field responder), so the answer should be a **deliberate architectural choice**, not a single default.

Recommendation: use both, for different roles — a Next.js command-centre web dashboard, plus a Flutter field-response mobile app. This is not a compromise; it mirrors how real disaster-response tooling is structured (a GIS-heavy ops console + a lightweight field app).

Layer	Choice	Why
Command-centre dashboard (district authority, control room)	Next.js (React) web app	Authorities work at a desk/laptop with large screens for the 100-zone grid map, severity heatmaps, resource-allocation approval gates, and multi-zone route visualisation. Next.js gives server-side rendering for fast load on shared/low-spec office machines, easy integration with mapping libraries (Leaflet/Mapbox), and a single codebase deployable instantly (Vercel) for judge demos.
Field responder app (boat/ambulance/rescue teams)	Flutter mobile app	Field teams are on phones, often with poor connectivity, and need: push notifications for new dispatch instructions, offline-first route caching, GPS-based turn-by-turn guidance, and one-tap audio playback of multilingual instructions. Flutter gives one codebase for Android + iOS, strong offline-storage support (Hive/SQLite), and native push notifications (Firebase Cloud Messaging) — which a PWA cannot match reliably in low-network flood zones.

Why not just one stack for both? A Next.js website (as a PWA) can technically send push notifications and cache data offline, but flood zones often have patchy 2G/3G. Flutter's native background services and local storage handle that much more reliably — and that reliability matters when lives are involved. A Flutter-only dashboard, on the other hand, is more effort than needed for a data-heavy, desk-based control-room screen. **For the hackathon demo, build Next.js first** (faster to build, easy to demo live in a browser) and present the Flutter app as the clear, reasoned next step — not a missing feature.

Complete Technology Stack

Layer	Technology	Purpose
Computer Vision	U-Net, YOLOv8, OpenCV, PyTorch, NumPy	Flood segmentation, damage/object detection
AI Orchestration	Google Gemini API, LangGraph, custom agents	Resource reasoning, multi-step decision workflows
Data & Storage	SQLite, JSON, Pandas, GeoPandas	Zone/victim/resource/route persistence
Geo-spatial & Routing	OpenStreetMap, NetworkX (Dijkstra)	Flood-aware shortest-path routing
Communication	Twilio API, gTTS, Google Translate API	SMS/WhatsApp alerts, multilingual audio instructions
Command-centre Frontend	Next.js, React, Tailwind CSS, Leaflet/Mapbox	Control-room dashboard, approval gates, map views
Field Frontend	Flutter, Firebase Cloud Messaging, Hive (offline cache)	Field app, push alerts, offline routes
Backend / API	FastAPI (Python) or Node.js (Express)	REST/WebSocket layer connecting AI pipeline to frontends

Layer	Technology	Purpose
Deployment	Docker, Vercel (web), Render/Railway or AWS (backend + ML)	Containerised, cloud-deployable for scale

07 Datasets & References

Real, citable datasets to train/benchmark the vision components and to point judges to when asked "what did you train on":

Dataset	Use in AEGIS	Link
xBD (building damage assessment)	Pre/post-disaster satellite image pairs with 4-class building damage labels across 19 disaster events (incl. floods) — trains/benchmarks the damage-detection (YOLO) stage.	xview2.org / arxiv.org/abs/1911.09296
FloodNet	High-resolution UAV imagery captured after Hurricane Harvey, pixel-labelled for flooded vs. non-flooded buildings/roads/water — directly benchmarks the U-Net flood-segmentation stage.	github.com/BinaLab/FloodNet-Supervised_v1.0
RescueNet	UAV post-disaster imagery with pixel-level segmentation across buildings, roads, pools, trees — useful for the drone-intelligence stage's scene understanding.	RescueNet, Scientific Data (Nature), 2023
Sen1Floods11	Sentinel-1/2 satellite flood-extent labels across 11 flood events globally — usable for satellite-scale flood-map generation when drone coverage isn't available.	github.com/cloudtostreet/Sen1Floods11
Bhuvan (ISRO/NRSC)	Indian satellite imagery & flood-mapping layers — the natural source for an India-specific pilot, aligning the solution with domestic geospatial infrastructure.	bhuvan.nrsc.gov.in
Copernicus EMS Rapid Mapping	Reference rapid-mapping methodology and open post-disaster imagery from EU's emergency mapping service — good for comparative validation.	emergency.copernicus.eu

Framework / API references (from the original deck, retained): U-Net paper (arxiv.org/abs/1505.04597) · Ultralytics YOLO docs (docs.ultralytics.com) · OpenStreetMap (openstreetmap.org) · NetworkX shortest-paths & docs (networkx.org) · Gemini API docs (ai.google.dev/gemini-api/docs) · Google Text-to-Speech (cloud.google.com/text-to-speech/docs) · Twilio docs (twilio.com/docs) · LangGraph (docs.langchain.com/oss/python/langgraph/overview) · UNDRR (undrr.org).

08 Language That Signals Rigour to Judges

Weave these terms naturally into the pitch and Q&A — they signal that the team understands the problem beyond a demo, which is what separates finalist decks from the rest:

- **Human-in-the-loop / human-on-the-loop** — for the two approval gates (shows responsible AI, not blind automation)
- **Explainable severity scoring** — the combined score is a transparent formula, not a black-box output
- **Graceful degradation** — system still functions with satellite-only input if drones can't fly
- **Interoperability with existing systems** — designed to sit alongside NDMA/SDMA workflows, not replace them
- **Federated / phased rollout** — start as a command-centre decision-support layer, scale zone-by-zone
- **Data provenance and auditability** — every AI recommendation and human approval is logged in SQLite for post-incident review
- **Low-bandwidth resilience** — SMS/audio fallback for field teams with no data connectivity
- **Multilingual, low-literacy accessible** — audio instructions via gTTS for field-level usability

09 Future Enhancements

The web dashboard (Next.js) is the main working build for the hackathon demo. The mobile app for field teams is planned as the next step, not built in full during the hackathon — shown below as a near-term item with reasoning, not a finished feature.

Horizon	Enhancement
Near-term (post-hackathon)	Flutter field app for rescue teams, built after the web dashboard. Reason: field teams work on phones with weak or no network, so they need offline-cached routes and native push alerts — things a website cannot reliably do in a flood zone. Also planned: live drone video feed instead of static images, and SMS-based two-way confirmation from field teams so the command centre knows an instruction was received.
Mid-term	A risk-prediction layer using past zone-severity data plus IMD/CWC weather forecasts, so resources can be placed before a flood peaks, not just after. Also: linking with NDMA's national disaster database, and adding citizen/crowd-sourced reports (from social media or a public app) alongside drone data for a fuller picture.
Long-term	Extending the same pipeline to other disaster types (earthquakes, cyclones, wildfires) using xBD's multi-hazard training data. Also: connecting multiple district command centres into one state-level dashboard, and running detection directly on the drone (edge inference) for faster results without waiting on a cloud connection.

10 Anticipated Judge Questions & Suggested Answers

Organised from introductory to advanced, including cross-questioning and background/team questions. Answers below are talking-point frameworks — adapt in your own words during the demo.

A. Introductory

Q: Tell us about your project.

A: AEGIS is an AI pipeline that turns a disaster image into a field-ready rescue dispatch in five stages — vision analysis, zone prioritisation, drone intelligence, AI resource decisioning, and safe routing — with two human-approval gates so authorities stay in control of every commitment of resources.

Q: What problem does it solve, in one line?

A: It compresses the time between 'a disaster has happened' and 'the right resource is moving down a safe route to the right place' — which today is a slow, manual, map-and-radio process.

Q: Why did you choose this problem statement?

A: PS 26206 is deliberately broad — risk mitigation, planning, and management across all disaster phases — which let us design for the highest-impact phase, response coordination, while keeping the architecture (zone database, severity scoring) reusable for pre-disaster planning too.

Q: Who is the end user?

A: Two user groups: district/state disaster-management authorities in a command centre (via the web dashboard), and field rescue teams — boat, ambulance, ground units — via mobile alerts.

B. Engineering & Technical Depth

Q: Walk us through the engineering pipeline end to end.

A: Image in → U-Net segments flood extent → YOLOv8 detects structural damage and objects → the area is split into a 10×10 grid and each zone gets a flood score + damage score combined into a severity score → top zones get drone passes for victim counts → Gemini reasons over severity + victim data to recommend resources → a human approves → Dijkstra over OpenStreetMap plans the safest route avoiding flagged-flooded roads → a second human approves → Twilio/gTTS dispatch the instruction.

Q: Why U-Net for segmentation and YOLO for detection instead of one model doing both?

A: They solve different problems well: U-Net's encoder-decoder structure is suited to dense pixel-wise segmentation (flood extent as a mask), while YOLO is optimised for fast bounding-box object detection (discrete damaged structures, people, vehicles). Using a single model for both would mean compromising on one task's accuracy — a real risk when the output feeds a resource-allocation decision.

Q: How do you compute the severity score, exactly?

A: It's a weighted combination of the flood score (proportion/depth of flooded pixels in a zone) and the damage score (count/severity of detected damaged structures), normalised per zone so zones are comparably ranked. We keep the formula transparent and inspectable rather than a learned black-box score, specifically so a human approver can sanity-check the ranking.

Q: Why Dijkstra and not A* or a learned routing model?

A: Dijkstra guarantees the shortest safe path on a graph where we can hard-block edges representing flooded/damaged roads — which is exactly the constraint we need, and it's deterministic and auditable, which matters for a life-safety system. A* would only help if we had a strong admissible heuristic; at city/district scale the performance gain isn't worth the added complexity for a first version.

Q: Why use an LLM (Gemini) in a safety-critical resource decision at all?

A: Gemini doesn't make the final call — it synthesises structured inputs (severity, victim counts, resource availability, disaster type) into a recommended allocation, which a human then approves or overrides at Approval Gate 1. It's a reasoning/summarisation layer over structured data, not an autonomous decision-maker.

Q: What happens if the AI is wrong — false positive/negative on damage detection?

A: Two mitigations: (1) both approval gates mean nothing is dispatched without human sign-off, so a bad AI call gets caught before real-world action; (2) the drone-intelligence stage exists precisely to re-verify top-priority zones at higher resolution before committing resources, catching satellite-level false positives.

Q: How does the system handle no/low connectivity in flood zones?

A: Dispatch falls back to SMS (Twilio) rather than requiring an app connection, and instructions are converted to audio (gTTS) for low-literacy or hands-busy field use. The planned Flutter app adds offline route caching so a team can navigate without a live connection once a route is downloaded.

C. Cross-Questioning & Scrutiny

Q: This looks like a lot for a hackathon timeframe — what's actually built vs. planned?

A: Be honest here: state clearly which pipeline stages are working prototype vs. stubbed/simulated for the demo (e.g., 'vision analysis and severity scoring run on real sample imagery; drone dispatch is simulated since we don't have physical drone access'). Judges respect a clear-eyed status update far more than an implied claim that everything is production-ready.

Q: How is this different from existing disaster-management platforms (e.g., NDMA's systems, Sahana, Ushahidi)?

A: Most existing platforms are information/coordination dashboards that rely on manual data entry. AEGIS's differentiation is the automated vision-to-severity pipeline that removes manual assessment from the critical path, plus the safety-aware routing layer — it's designed to plug into those systems as an intelligence layer, not replace them.

Q: What's your accuracy on flood/damage detection, and where does it break down?

A: Cite the benchmark ranges from the datasets you train/validate on (e.g., U-Net-style flood segmentation on FloodNet typically reports high mean-IoU in the 70-80%+ range in published work; give your own measured number if you have one). Be upfront about known failure modes — e.g., distinguishing natural water bodies from floodwater, and detecting damage under cloud cover — since FloodNet's own authors flag these as open challenges.

Q: Who is accountable if a resource is misallocated because of an AI recommendation?

A: The human approver at the gate — the system is explicitly designed so accountability stays with the authority who clicks approve, and every recommendation + decision is logged in the SQLite audit trail for post-incident review.

Q: How would this scale beyond a demo — cost, compute, data availability?

A: Satellite/drone imagery access and Gemini API calls are the main recurring costs; the architecture is stateless per-zone so it scales horizontally, and starting as a command-centre decision-support tool (not a full replacement system) lets it be piloted in one district before wider rollout.

D. Background, Team & Reflection

Q: What was the hardest technical decision your team made?

A: Frame around a real trade-off you made — e.g., choosing rule-based Dijkstra routing over a learned routing model for auditability, or choosing a transparent severity formula over a black-box classifier so human approvers could trust and override it.

Q: How did you divide work across the team?

A: Answer honestly based on your actual team split (e.g., CV/ML, backend/orchestration, frontend/dashboard, research/domain). Judges are checking for real ownership, not a rehearsed line.

Q: What would you do differently if you started over?

A: A good answer shows reflection without undermining the project — e.g., 'we'd validate the severity scoring formula against real disaster-response SOPs earlier, rather than designing it in isolation and validating afterward.'

Q: Why should this idea get funded/selected over other disaster-management submissions?

A: Anchor on the combination that's hard to replicate quickly: an end-to-end pipeline (not just one model), human-in-the-loop safety design, and low-bandwidth field delivery — rather than claiming to be the only team with AI-based flood detection.

11 Feasibility, Viability & Scale — Summary

Feasibility: uses information that's already available during emergencies (imagery, affected-area data). Humans stay in control at both approval gates. The current prototype already shows the full journey from detection to a planned response.

Viability: cuts the time and effort needed to find priorities and coordinate resources. It can start as a decision-support layer for the command centre, without replacing any existing emergency system. It also adapts to different flood-prone regions and different incident sizes.

Sustainability & scale: starts small, with one focused use case (flood response), and grows as more operational data comes in. It works alongside existing rescue teams and authorities rather than around them. Future integration with live field updates will make the response keep adapting in real time.

Prepared as a supporting dossier for Team Diamonds — Smart India Hackathon 2026, Problem Statement 26206. This document is for internal team preparation and does not replace the official SIH PPT template submission.